Lire un message d'erreur

Où regarder, ce que dit chaque partie, quoi faire

Kevin Bardoux

Table des matières

La méthode, pour tous les outils 1
Node.js (semaine 2) 2
Une variable qui n'existe pas : `ReferenceError` 2
Une opération impossible : `TypeError` 2
Une faute de syntaxe : `SyntaxError` 3
Un fichier introuvable : `Cannot find module` 3
Un test rouge : `node --test` 4
Tout le fichier de tests échoue : `tests 1`, `fail 1` 4
ESLint (semaine 2) 5
ESLint ne souligne rien dans VS Code 5
Git (semaine 4) 6
Pas dans un dépôt 6
`git log` ne montre rien 6
Un conflit de fusion 7
Un envoi refusé 7
Un avertissement de fin de ligne (Windows) 7
Sources 8

Un message d'erreur n'est pas un reproche : c'est une indication précise, qui dit **quoi**, **où** et souvent **pourquoi**. Cette fiche montre de vrais messages, outil par outil. Elle grandit avec la formation : une section y entre avec la semaine de son outil.

La méthode, pour tous les outils

1. **Lisez le type** : le premier mot important (`TypeError`, `error`, `fatal`, `CONFLICT`…).

2. **Lisez le message** : la phrase qui suit, mot par mot. Cherchez les mots inconnus.
3. **Trouvez l'endroit** : le fichier, la ligne, parfois la colonne.
4. **Corrigez une chose à la fois**, puis relancez.

C'est la démarche de débogage de la semaine 3 : lire, isoler, tester, corriger. Les chemins des fichiers affichés ici viennent d'un ordinateur sous Linux ; sous Windows, ils commencent par `C:/Users/…`. Certains outils affichent leurs messages en français : le sens et la place de chaque partie restent les mêmes.

Node.js (semaine 2)

Une variable qui n'existe pas : `ReferenceError`

```
file:///…/dwwm/semaine-02/ref.js:1
console.log(prix);
            ^

ReferenceError: prix is not defined
    at file:///…/dwwm/semaine-02/ref.js:1:13
```

- **Où** : la première ligne donne le fichier et la ligne (`ref.js:1`) ; le `^` montre l'endroit exact.
- **Type** : `ReferenceError`, une variable utilisée alors qu'elle n'existe pas (MDN Web Docs, s. d.-a).
- **Message** : `prix is not defined`, « `prix` n'est pas défini ».
- **Quoi faire** : vérifiez l'orthographe du nom, et que la variable est déclarée (`const`, `let`) **avant** cette ligne, dans le même bloc.

Une opération impossible : `TypeError`

```
file:///…/dwwm/semaine-02/type.js:2
console.log(word.length);
                ^

TypeError: Cannot read properties of undefined (reading
'length')
    at file:///…/dwwm/semaine-02/type.js:2:18
```

- **Type** : `TypeError`, une opération impossible sur une valeur du mauvais type (MDN Web Docs, s. d.-b).
- **Message** : « impossible de lire les propriétés de `undefined` (en lisant `length`) » : `word` vaut `undefined`.
- **Quoi faire** : cherchez pourquoi la valeur vaut `undefined` (un argument oublié, une variable jamais remplie), puis traitez ce cas d'abord (style défensif).

Une faute de syntaxe : `SyntaxError`

```
file:///…/dwwm/semaine-02/syn.js:1
console.log('bonjour';
            ^^^^^^^^^

SyntaxError: missing ) after argument list
```

- **Type** : `SyntaxError`, le code ne respecte pas la grammaire de JavaScript (MDN contributors, s. d.) : le programme ne démarre même pas.
- **Message** : « il manque `)` après la liste des arguments ».
- **Quoi faire** : regardez la ligne indiquée **et** la précédente ; comptez les parenthèses, les accolades et les guillemets ouverts et fermés.

Un fichier introuvable : `Cannot find module`

```
Error: Cannot find module '/…/dwwm/semaine-02/
absent.js'
```

- **Message** : Node ne trouve pas le fichier demandé (OpenJS Foundation, s. d.-a).
- **Quoi faire** : tapez `pwd`, puis `ls` : êtes-vous dans le bon dossier ? Le nom est-il exact, extension comprise (pas `absent.js.txt`) ?

Sous chaque erreur, les lignes qui commencent par `at` forment la **pile d'appels** : la première ligne qui mentionne **votre** fichier est la plus utile ; les lignes `node:internal/…` concernent Node lui-même.

Un test rouge : `node --test`

```
✖ affiche Hello, world!
  AssertionError [ERR_ASSERTION]: Expected values to be
strictly equal:
  + actual - expected

  + 'À faire'
  - 'Hello, world!'
```

- **Où** : le nom du test qui échoue, après ✖.
- **`+ actual`** : ce que votre programme affiche ou renvoie ; **`- expected`** : ce qu'il devrait afficher (OpenJS Foundation, s. d.-b).
- **Quoi faire** : comparez les deux lignes caractère par caractère (espaces, majuscules, ponctuation), corrigez, relancez.

Tout le fichier de tests échoue : `tests 1`, `fail 1`

```
file:///…/aire.js:2
  let width = 3;
      ^

SyntaxError: Identifier 'width' has already been
declared
    at …
✖ aire.test.js
ℹ tests 1
ℹ pass 0
ℹ fail 1
```

- **Où** : après ✖, le nom du **fichier** de tests, pas celui d'un test ; et `tests 1` alors que ce fichier en contient plusieurs.
- **Message** : votre fichier ne s'est pas chargé, aucun test n'a pu tourner (OpenJS Foundation, s. d.-b). La cause est écrite au-dessus : une erreur de syntaxe (ici, un paramètre déclaré une deuxième fois), ou une ligne placée hors d'une fonction qui plante au chargement.

- **Quoi faire** : lancez `node votre-fichier.js` pour voir l'erreur seule ; lisez la ligne indiquée et le `^` ; essayez vos fonctions dans un fichier `essai.js` séparé, jamais dans le fichier testé.

ESLint (semaine 2)

```
 8:1   error  Unexpected var, use let or const instead
no-var
12:15  error  Expected '===' and instead saw '=='
eqeqeq
17:13  error  'mesage' is not defined
no-undef

✖ 5 problems (5 errors, 0 warnings)
```

- **Où** : `ligne:colonne` en début de ligne (`12:15` : ligne 12, colonne 15).
- **Message** : ce qui ne va pas, en anglais.
- **Règle** : le dernier mot (`eqeqeq`) ; il mène à sa page de documentation (ESLint, s. d.).
- **Quoi faire** : corrigez une alerte à la fois, en partant du haut, puis relancez `npx eslint .`.

ESLint ne souligne rien dans VS Code

Pas de message ici : c'est l'absence de vague rouge qui alerte, alors que le fichier contient une erreur (par exemple `var x = 1;`). Vérifiez dans l'ordre :

1. **`npm install` a-t-il été lancé dans ce dossier ?** Un dossier `node_modules` doit être à côté de `package.json`. Sans lui, `npm run lint` dans le terminal se termine par une ligne qui dit que la commande `eslint` est introuvable (son libellé dépend du système ; sous Linux : `sh: 1: eslint: not found`). Lancez `npm install` dans le dossier de la semaine.
2. **Le bon dossier est-il ouvert ?** VS Code doit avoir ouvert le dossier de la semaine (celui qui contient `eslint.config.js`), pas `dwwm` tout entier ni un fichier seul : *Fichier*, *Ouvrir le dossier…*.

3. **L'extension ESLint est-elle installée et activée ?** Vue Extensions (`Cmd+Shift+X` (mac), `Ctrl+Shift+X` (windows)) : cherchez ESLint (éditeur Microsoft) ; le bouton doit proposer *Désactiver*, pas *Installer* ni *Activer*.
4. **Toujours rien ?** Tapez `npx eslint nom-du-fichier.js` dans le terminal. S'il affiche les erreurs, ESLint marche et c'est l'éditeur qui ne l'a pas encore chargé : fermez puis rouvrez VS Code. S'il n'affiche rien non plus, appelez le formateur.

Git (semaine 4)

Pas dans un dépôt

```
fatal: not a git repository (or any of the parent
directories): .git
```

- **Message** : le dossier courant n'est pas un dépôt Git, ni aucun dossier parent.
- **Quoi faire** : tapez `pwd` ; placez-vous dans le dossier du projet (`cd …`), ou créez le dépôt avec `git init` s'il n'existe pas encore.

`git log` ne montre rien

```
fatal: votre branche actuelle 'main' ne contient encore
aucun commit
```

En anglais : `fatal: your current branch 'main' does not have any commits yet`. Avec `--graph --all`, `git log` n'affiche rien du tout, sans message.

- **Message** : le dossier est bien un dépôt Git, mais aucun commit n'y est encore enregistré.
- **Quoi faire** : tapez `pwd` : êtes-vous dans le bon dossier (par exemple `~/dwwm/git-journal`) ? Un `git init` lancé dans un autre dossier crée un second dépôt, vide. Dans le bon dossier, tapez `git status` : des fichiers en rouge ou en vert attendent `git add`, puis `git commit -m "…"`. Si le message est `not a git repository`, voir « Pas dans un dépôt ».

Un conflit de fusion

```
Auto-merging journal.md
CONFLICT (content): Merge conflict in journal.md
Automatic merge failed; fix conflicts and then commit
the result.
```

Le même message, quand Git s'affiche en français :

```
Fusion automatique de journal.md
CONFLIT (contenu) : Conflit de fusion dans journal.md
La fusion automatique a échoué ; réglez les conflits et
validez le résultat.
```

- **Où** : le fichier en conflit (`journal.md`).
- **Message** : les deux branches ont modifié les mêmes lignes ; Git ne choisit pas.
- **Quoi faire** : ouvrez le fichier, gardez le bon texte entre les marqueurs `<<<<<<<`, `=======` et `>>>>>>>`, effacez les marqueurs, puis `git add` et `git commit` (GitHub, s. d.-b).

Un envoi refusé

```
 ! [rejected]        HEAD -> main (fetch first)
error: failed to push some refs to '…'
hint: Updates were rejected because the remote contains
work that you do
hint: not have locally.
```

- **Message** : le dépôt distant contient des commits que vous n'avez pas.
- **Quoi faire** : récupérez-les d'abord avec `git pull`, réglez un éventuel conflit, puis relancez `git push` (Scott Chacon et Ben Straub, s. d.). Jamais de `git push --force` sans l'accord du formateur.

Un avertissement de fin de ligne (Windows)

```
warning: in the working copy of 'notes.md', LF will be
replaced by CRLF the next time Git touches it
```

- **Où** : après `git add`, sous Windows, pour un fichier créé dans Git Bash (par exemple avec `echo`).
- **Message** : ce n'est pas une erreur. Windows termine les lignes autrement que macOS et Linux (CRLF au lieu de LF) ; Git convertit tout seul (GitHub, s. d.-a). Le fichier est bien ajouté.
- **Quoi faire** : rien ; continuez avec `git commit`. Cet avertissement reste en anglais, même quand Git s'affiche en français.

Sources

ESLint. s. d. « Getting Started with ESLint ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/use/getting-started.

GitHub. s. d.-a. « Configuring Git to handle line endings ». Consulté le 17 octobre 2026. https://docs.github.com/en/get-started/git-basics/configuring-git-to-handle-line-endings.

GitHub. s. d.-b. « Resolving a merge conflict using the command line ». Consulté le 26 septembre 2026. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line.

MDN contributors. s. d. « SyntaxError — JavaScript | MDN ». Consulté le 3 octobre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/SyntaxError.

MDN Web Docs. s. d.-a. « ReferenceError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/ReferenceError.

MDN Web Docs. s. d.-b. « TypeError - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/TypeError.

OpenJS Foundation. s. d.-a. « Errors — Node.js Documentation ». Consulté le 3 octobre 2026. https://nodejs.org/api/errors.html.

OpenJS Foundation. s. d.-b. « Test runner | Node.js Documentation ». Consulté le 26 septembre 2026. https://nodejs.org/api/test.html.

Scott Chacon et Ben Straub. s. d. « Pro Git : Travailler avec des dépôts distants ». Consulté le 26 septembre 2026. https://git-scm.

com/book/fr/v2/Les-bases-de-Git-Travailler-avec-des-d%C3%A9p%C3%B4ts-distants.